Diseño de un diagrama de clases UML. (CASO PRÁCTICO). UD04

Entornos de desarrollo

Silvio aguilar

1. INTRODUCCIÓN

Este diagrama de clases UML representa un sistema simplificado de gestión de pedidos online. El sistema permite gestionar usuarios (clientes y administradores), productos, pedidos y los detalles de cada pedido. El diseño sigue los principios de la programación orientada a objetos, utilizando herencia, asociación y composición para modelar las relaciones entre las entidades del sistema.

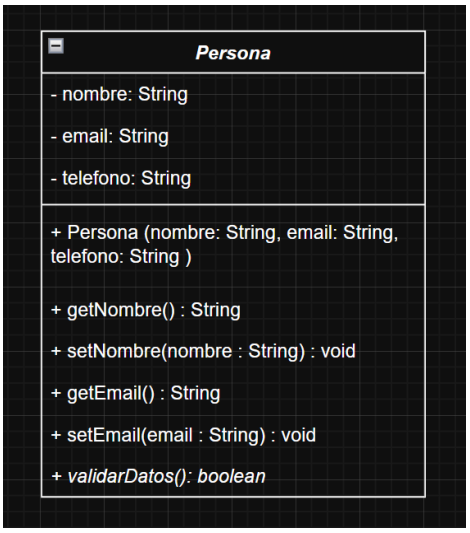
2. DESCRIPCIÓN DE CLASES

2.1 Clase Persona (abstracta)

Clase base abstracta que representa a cualquier individuo en el sistema.

Atributos:
- nombre : String - Nombre completo de la persona
- email : String - Dirección de correo electrónico
- telefono : String - Número de teléfono de contacto

Métodos:
+ Persona(nombre : String, email : String, telefono : String) - Constructor
+ getNombre() : String - Retorna el nombre
+ setNombre(nombre : String) : void - Establece el nombre
+ getEmail() : String - Retorna el email
+ setEmail(email : String) : void - Establece el email
+ validarDatos() : boolean - Método abstracto para validar datos (implementado por clases hijas)

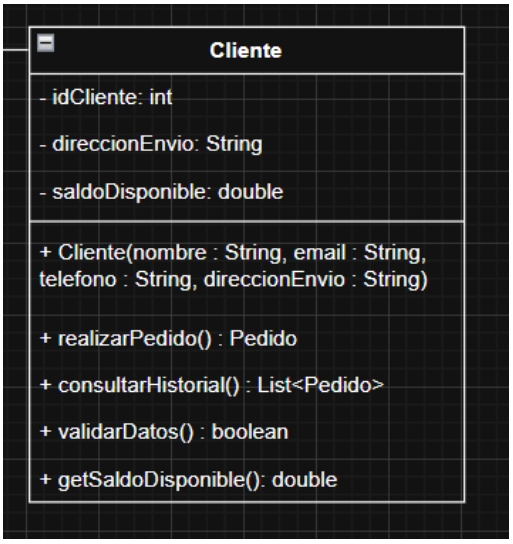


2.2 Clase Cliente

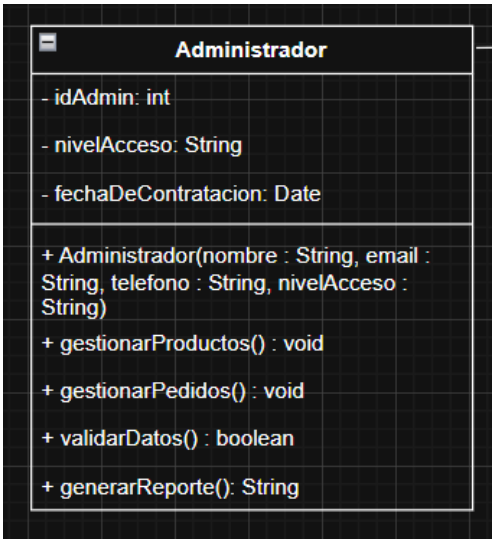
Representa a los clientes que pueden realizar pedidos.

Atributos:
- idCliente : int - Identificador único del cliente
- direccionEnvio : String - Dirección para envío de pedidos
- saldoDisponible : double - Saldo disponible del cliente

Métodos:
+ Cliente(...) - Constructor con parámetros heredados y propios
+ realizarPedido() : Pedido - Crea un nuevo pedido
+ consultarHistorial() : List<Pedido> - Retorna historial de pedidos
+ validarDatos() : boolean - Implementación de validación específica
+ getSaldoDisponible() : double - Retorna el saldo disponible



2.3 Clase Administrador



Representa a los administradores del sistema.

Atributos:
- idAdmin : int - Identificador único del administrador
- nivelAcceso : String - Nivel de privilegios del administrador
- fechaContratacion : Date - Fecha de contratación

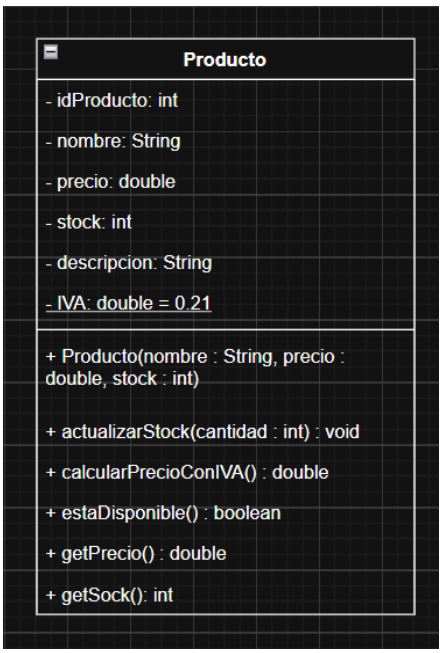
Métodos:
+ Administrador(...) - Constructor
+ gestionarProductos() : void - Permite gestionar el catálogo
+ gestionarPedidos() : void - Permite administrar pedidos
+ validarDatos() : boolean - Implementación de validación específica
+ generarReporte() : String - Genera reportes del sistema

2.4 Clase Producto

Representa los productos disponibles en el sistema.

Atributos:
- idProducto : int - Identificador único
- nombre : String - Nombre del producto
- precio : double - Precio unitario
- stock : int - Cantidad disponible en inventario
- descripcion : String - Descripción del producto
~ IVA : double = 0.21 - Constante con el porcentaje de IVA (subrayado por ser estático)

Métodos:
+ Producto(...) - Constructor
+ actualizarStock(cantidad : int) : void - Modifica el stock
+ calcularPrecioConIVA() : double - Calcula precio con IVA incluido
+ estaDisponible() : boolean - Verifica disponibilidad
+ getPrecio() : double - Retorna el precio
+ getStock() : int - Retorna el stock actual



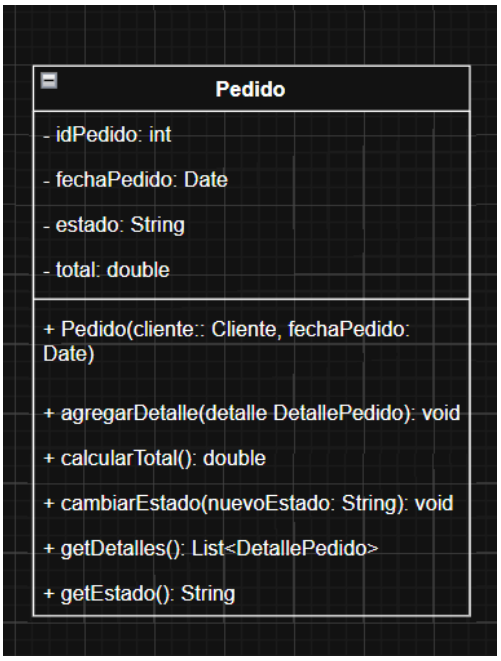
2.5 Clase Pedido

Representa un pedido realizado por un cliente.

Atributos:
- idPedido : int - Identificador único
- fechaPedido : Date - Fecha de creación

- estado : String - Estado actual (pendiente, enviado, entregado)
- total : double - Monto total del pedido

Métodos:
+ Pedido(...) - Constructor
+ agregarDetalle(detalle : DetallePedido) : void - Añade línea al pedido
+ calcularTotal() : double - Calcula el total sumando detalles
+ cambiarEstado(nuevoEstado : String) : void - Actualiza el estado
+ getDetalles() : List<DetallePedido> - Retorna los detalles
+ getEstado() : String - Retorna el estado actual

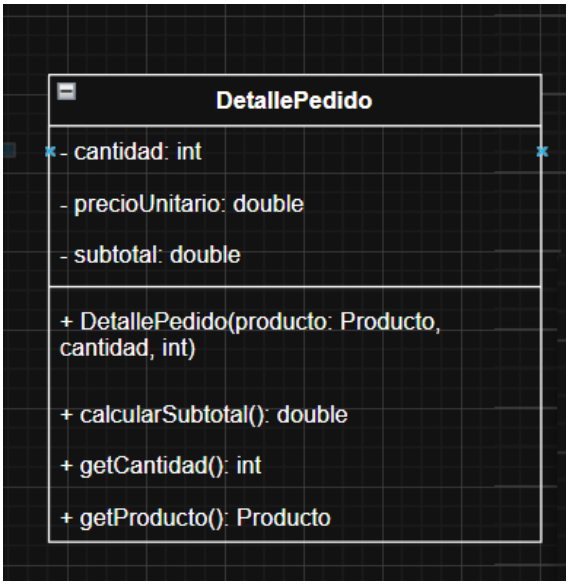


2.6 Clase DetallePedido

Representa una línea de pedido (producto + cantidad).

Atributos:
- cantidad : int - Cantidad del producto
- precioUnitario : double - Precio al momento del pedido
- subtotal : double - Resultado de cantidad × precio

Métodos:
+ DetallePedido(...) - Constructor
+ calcularSubtotal() : double - Calcula cantidad × precio
+ getCantidad() : int - Retorna la cantidad
+ getProducto() : Producto - Retorna el producto asociado



3. RELACIONES ENTRE CLASES

3.1 Herencia (Generalización)

Las clases Cliente y Administrador heredan de Persona.

Representación: Línea sólida con flecha hueca apuntando a la clase padre.

3.2 Asociación

a) Cliente → Pedido (1 a 0..*)

Un cliente puede realizar ninguno o varios pedidos, pero cada pedido pertenece a un único cliente.

Representación: Línea sólida con multiplicidad 1 en Cliente y 0..* en Pedido.
Nombre de relación: "realiza"
Navegabilidad: Desde Cliente hacia Pedido (el cliente conoce sus pedidos).

b) DetallePedido → Producto (* a 1)

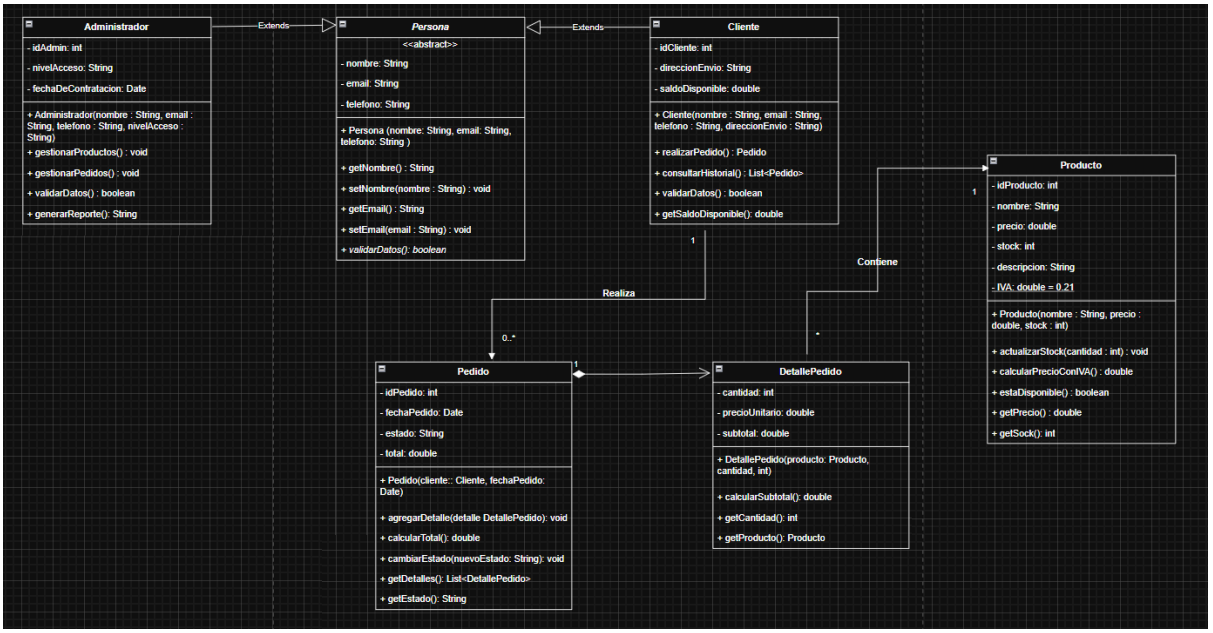
Cada DetallePedido está asociado a un único Producto, pero un Producto puede estar en múltiples DetallePedido.

Representación: Línea sólida con multiplicidad * en DetallePedido y 1 en Producto.
Nombre de relación: "contiene"
Navegabilidad: Desde DetallePedido hacia Producto.

3.3 Composición

Pedido ◆→ DetallePedido (1 a 1..*)

Relación de composición fuerte entre Pedido y DetallePedido.

Representación: Línea sólida con diamante negro sólido (◆) en Pedido.
Multiplicidad: 1 en Pedido, 1..* en DetallePedido (un pedido debe tener al menos un detalle).

4. DIAGRAMA COMPLETO